Data Preprocessing Lab

A credit card company wants to develop a model based on the history of data (Data.CSV).

Country	Age	Salary	Purchased
UAE	44	72000	No
Bahrain	27	48000	Yes
KSA	30	54000	No
Bahrain	38	61000	No
KSA	40		Yes
UAE	35	58000	Yes
Bahrain		52000	No
UAE	48	79000	Yes
KSA	50	83000	No
UAE	37	67000	Yes

Before developing a model, it has to pre-process the historical data. Pre-processing the historical data involves several steps, such as:

1. Import libraries: Import the necessary libraries for data manipulation and preprocessing, such as pandas, numpy, and scikit-learn.

2. Load data: Load the dataset into a suitable data structure, such as a pandas DataFrame or a numpy array.

3. Data cleaning: Handle missing values using techniques like mean imputation.

4. Data transformation: Transform categorical data into numerical representations using techniques like one-hot encoding or label encoding.

5. Normalization and scaling: Normalize or scale numerical features to a common range, ensuring that all features have equal influence on the learning algorithm.

Save preprocessed data: Save the preprocessed data for future use or for sharing with others.

Lab Task - 1

Perform the preprocessing for Data.CSV

a.	Importing the libraries

	`import numpy as np`
	`import matplotlib.pyplot as plt`
	`import pandas as pd`

b.	Importing the dataset

```
dataset = pd.read_csv('Data.csv')
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values
print(X)
print(y)
```

c. Taking care of missing data

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(missing_values=np.nan, strategy='mean')

X[:, 1:3] = imputer.fit_transform(X[:, 1:3])
```

```
(or)
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(missing_values=np.nan, strategy='mean')
imputer.fit(X[:, 1:3])
X[:, 1:3] = imputer.transform(X[:, 1:3])
print(X)
```

d. Encoding the Independent Variable

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [0])], remainder='passthrough')
# X = np.array(ct.fit_transform(X))
X = ct.fit_transform(X)
```

```
(or)
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(),
[0])], remainder='passthrough')
X = np.array(ct.fit_transform(X))
print(X)
```

e. Encoding the Dependent Variable

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y = le.fit_transform(y)
print(y)
```

f. Splitting the dataset into the Training set and Test set

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size = 0.2, random_state = 1)
print(X_train)
print(X_test)
print(y_train)
print(y_test)
```

g. Feature Scaling

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
```

```
X_train[:,3:]= sc.fit_transform(X_train[:,3:])
X_test[:,3:]= sc.transform(X_test[:,3:])
print(X_train)
print(X_test)
```

Do it yourself: Perform data preprocessing on the 'Marks.xlsx' dataset where 'Total' marks is the target variable and 'Quiz', 'Project', 'Practical' and 'Final' exams are the independent variables. Use filtering to extract only these relevant columns and exclude 'id', 'division', and other unnecessary fields.

Steps to be Performed:

1. Loaded the dataset `Marks.csv`.
2. Selected relevant columns: `Quiz`, `Practical`, `Project`, and `Final`.
3. Filled missing values with the mean of respective columns.
4. Created a new column `Total` as the weighted sum:
   - `10%` of `Quiz`
   - `30%` each for `Practical`, `Project`, and `Final`
5. Saved the pre-processed dataset as `Preprocessed_Marks.csv`.

```
import pandas as pd

# Load the dataset
marks_file_path = 'Marks.csv'
marks_data = pd.read_csv(marks_file_path)

# Select only relevant columns: Quiz, Practical, Project, Final
filtered_data = marks_data[['Quiz', 'Practical', 'Project', 'Final']]

# Fill missing values with the mean of respective columns (if any)
filtered_data.fillna(filtered_data.mean(), inplace=True)

# Create a new column 'Total' as the target variable
filtered_data['Total'] = (
    0.1 * filtered_data['Quiz'] +
    0.3 * filtered_data['Practical'] +
    0.3 * filtered_data['Project'] +
    0.3 * filtered_data['Final']
)

# Save the preprocessed dataset
filtered_data.to_csv('Preprocessed_Marks.csv', index=False)

# Display the first few rows of the preprocessed dataset
print(filtered_data.head())
```